PRACTICAL – 1

Practical – 1 (a): Create a College Information Portal PWA

Step 1: Open Index.cshtml

1. Open Visual Studio.
2. Open your ASP.NET Core project.
3. Go to Views → Home → Index.cshtml.
4. Write the following code:

```
<div class="text-center">
  <h1>College Information Portal</h1>
  <h3>Welcome to XYZ College</h3>
  <p>Courses Offered:</p>
  <ul>
    <li>BSc IT</li>
    <li>BSc CS</li>
    <li>BCA</li>
  </ul>
  <p>Contact: 9876543210</p>
</div>
```

Step 2: Create service-worker.js

1. Open the wwwroot folder.
2. Right-click the folder and select Add → New Item.
3. Create a JavaScript file named service-worker.js.
4. Write the following code:

```
self.addEventListener('install', function (event) {
    console.log('Service Worker Installed');
});

self.addEventListener('fetch', function (event) {
    console.log('Fetching:', event.request.url);
});
```

Step 3: Create manifest.json

1. Right-click the wwwroot folder.
2. Select Add → New Item.
3. Create a JSON file named manifest.json.

4. Write the following code:

```
{
  "name": "College Information Portal",
  "short_name": "CollegePortal",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any"
    }
  ]
}
```

Step 4: Create the Icons Folder

1. Right-click wwwroot.
2. Select Add → New Folder.
3. Name the folder icons.
4. Add icon1.png with dimensions 192 × 192 pixels.
5. Add icon2.png with dimensions 512 × 512 pixels.

Step 5: Update Layout.cshtml

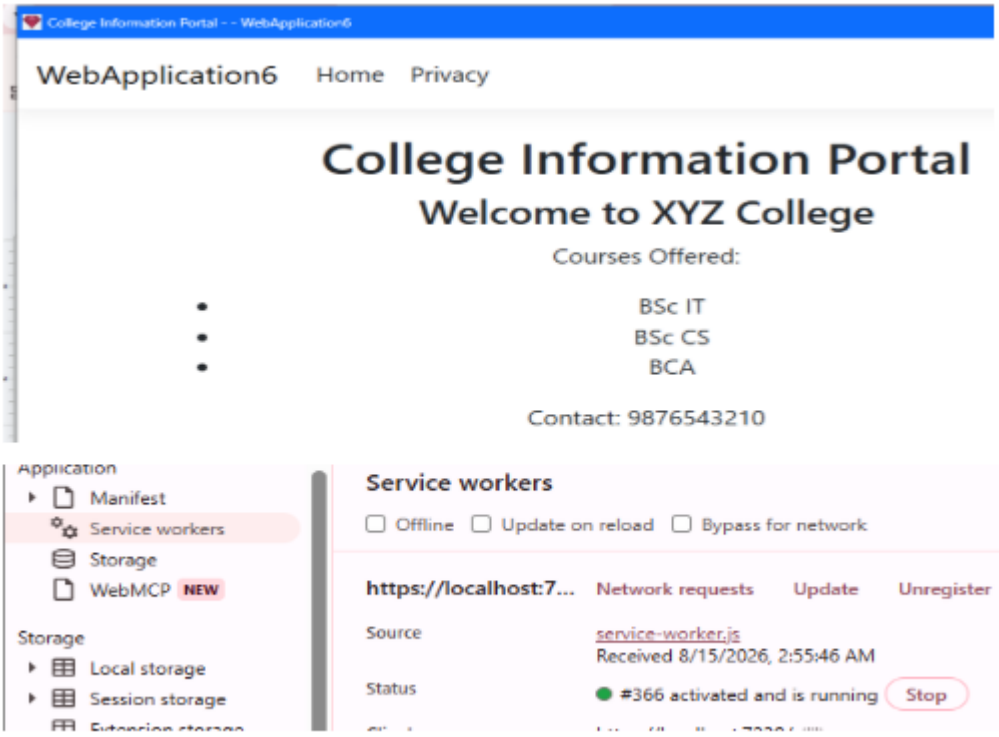
1. Open Views → Shared → _Layout.cshtml.
2. Find the <head> section.
3. Add the following lines inside <head>:

```
<link rel="manifest" href="/manifest.json">
<meta name="theme-color" content="#0d6efd">
```

Step 6: Run the Application

1. Save all files by pressing Ctrl + S.
2. Run the project in Visual Studio.
3. Open the application in your browser.
4. Check that the college name, courses and contact number are displayed

OUTPUT


College Information Portal - - WebApplication6
WebApplication6
Home
Privacy
College Information Portal
Welcome to XYZ College
Courses Offered:
BSc IT
BSc CS
BCA
Contact: 9876543210
Application
Manifest
Service workers
Storage
WebMCP
NEW
Storage
Local storage
Session storage
Service workers
Offline
Update on reload
Bypass for network
https://localhost:7...
Network requests
Update
Unregister
Source
service-worker.js
Received 8/15/2026, 2:55:46 AM
Status
#366 activated and is running
Stop

Practical – 1 (b): Create an Offline Web Application

Step 1: Open Index.cshtml

1. Open your ASP.NET Core project in Visual Studio.
2. Go to Views → Home → Index.cshtml.
3. Add the following code:

```
<div class="text-center">
  <h1>College Information Portal</h1>
  <h3>Welcome to XYZ College</h3>
  <p>This website supports Offline Mode.</p>
</div>
```

Step 2: Create offline.html

1. Open the wwwroot folder.
2. Create a new HTML file named offline.html.
3. Add the following code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Offline</title>
</head>
<body>
  <h1>No Internet Connection</h1>
  <h3>Please check your internet connection.</h3>
</body>
</html>
```

Step 3: Create service-worker.js

1. Inside the wwwroot folder, create a JavaScript file named service-worker.js.
2. Add the following code:

```
const CACHE_NAME = "offline-cache-v1";

const urlsToCache = [
    "/",
    "/offline.html"
];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
        .then(cache => {
            return cache.addAll(urlsToCache);
        })
    );
});
```

```
self.addEventListener("fetch", event => {
   event.respondWith(
      fetch(event.request)
      .catch(() => {
         return caches.match("/offline.html");
      })
   );
});
```

Step 4: Register the Service Worker in Layout.cshtml

1. Open Views → Shared → _Layout.cshtml.
2. Add the following code inside the <body> section, preferably before </body>:

```
<script>
   if ('serviceWorker' in navigator) {
      navigator.serviceWorker.register('/service-worker.js')
         .then(registration => {
            console.log(
               'Service Worker Registered:',
               registration
            );
         })
         .catch(error => {
            console.error(
               'Service Worker Registration Failed:',
               error
            );
         });
   }
</script>
```

Step 5: Configure Program.cs

1. Open the Program.cs file.
2. Add app.UseStaticFiles(); before app.UseRouting();.
3. Arrange the code as follows:

```
app.UseStaticFiles();

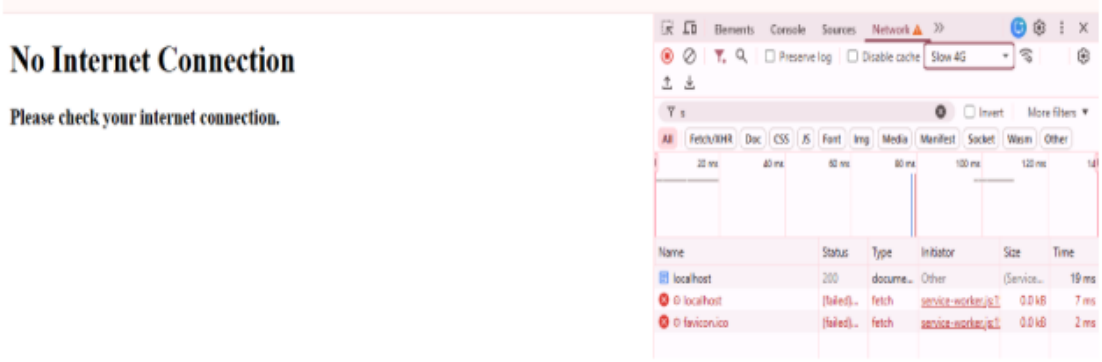
app.UseRouting();

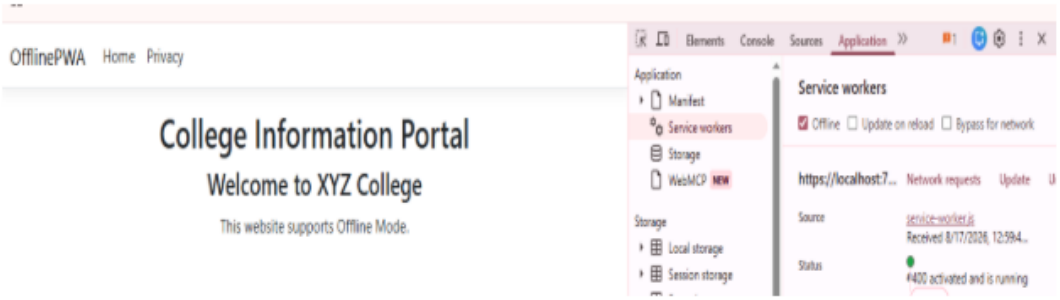
app.UseAuthorization();

app.MapControllerRoute(
  name: "default",
  pattern: "{controller=Home}/{action=Index}/{id?}");
```

Step 6: Run the Project

OUTPUT


No Internet Connection
Please check your internet connection.
Elements
Console
Sources
Network
Slow 4G
Name
Status
Type
Initiator
Size
Time
localhost
200
Other
19 ms
favicon.ico
7 ms
2 ms


OfflinePWA
Home
Privacy
College Information Portal
Welcome to XYZ College
This website supports Offline Mode.
Elements
Console
Sources
Application
Application
Manifest
Service workers
Storage
WebMCP
NEW
Storage
Local storage
Session storage
Service workers
Offline
Update on reload
Bypass for network
Network requests
Update
Source
Status
activated and is running

Practical – 2 (a): Create a PWA with Offline Caching

Step 1: Open Index.cshtml

1. Open your ASP.NET Core project in Visual Studio.
2. Go to Views → Home → Index.cshtml.
3. Add the following code:

```
<div class="text-center">
  <h1>Hello College Information Portal</h1>
  <h3>PWA Offline Demo</h3>
  <p>This application works without internet connection.</p>
</div>
```

Step 2: Create manifest.json

1. Open the wwwroot folder.
2. Create a new file named manifest.json.
3. Add the following code:

```
{
   "name": "Offline PWA",
   "short_name": "OfflinePWA",
   "start_url": "/",
   "display": "standalone",
   "background_color": "#ffffff",
   "theme_color": "#0d6efd",
   "icons": [
     {
       "src": "/icons/icon1.png",
       "sizes": "192x192",
       "type": "image/png"
     },
     {
       "src": "/icons/icon2.png",
       "sizes": "512x512",
       "type": "image/png"
     }
   ]
}
```

Step 3: Create service-worker.js

1. Inside the wwwroot folder, create a JavaScript file named service-worker.js.
2. Add the following code:

```
const CACHE_NAME = "pwa-cache-v1";

const urlsToCache = [
  "/",
  "/Home/Index"
```

```
];

self.addEventListener("install", event => {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => {
        return cache.addAll(urlsToCache);
      })
  );
});

self.addEventListener("fetch", event => {
  event.respondWith(
    caches.match(event.request)
      .then(response => {
        return response || fetch(event.request);
      })
  );
});
```

Step 4: Configure Program.cs

1. Open the Program.cs file.
2. Add app.UseStaticFiles(); before app.UseRouting();.

```
app.UseStaticFiles();

app.UseRouting();
```

Step 5: Register the Service Worker in Layout.cshtml

1. Open Views → Shared → _Layout.cshtml.
2. Add the following code before the closing </body> tag:

```
<script>
 if ('serviceWorker' in navigator) {

   window.addEventListener('load', function () {

       navigator.serviceWorker.register('/service-worker.js')
         .then(function (registration) {
           console.log(
             'Service Worker Registered:',
             registration.scope
           );
         })
         .catch(function (error) {
           console.error(
             'Service Worker Registration Failed:',
             error
```

```
            );
        });

    });

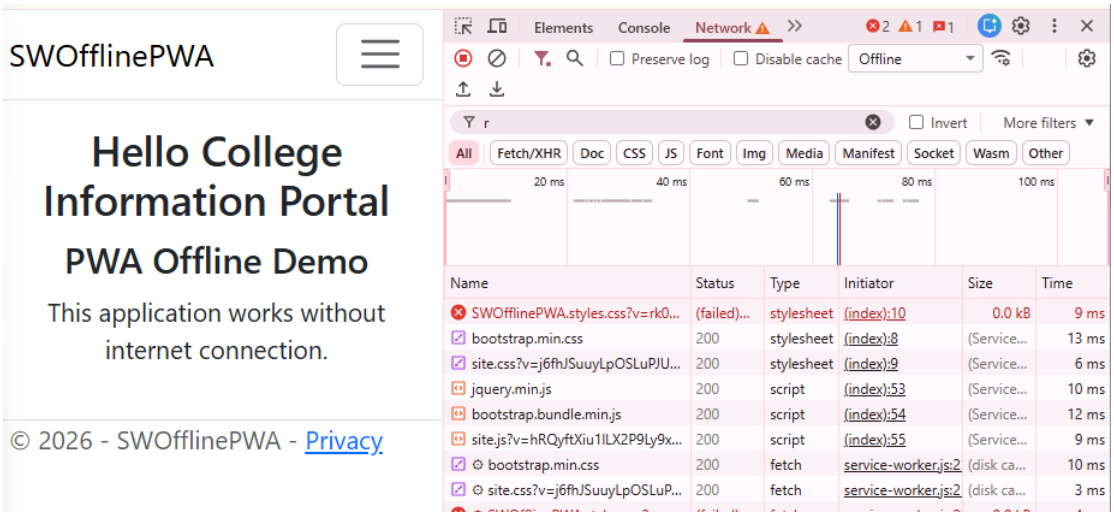
  }
</script>
```

Step 6: Create the Icons Folder

1. Inside wwwroot, create a folder named icons.
2. Add icon1.png with dimensions 192 × 192.
3. Add icon2.png with dimensions 512 × 512.

Step 7: Run the Project

OUTPUT



Practical – 2 (b): Create a Student Record PWA

Step 1: Create Student.cs

1. Open your ASP.NET Core project in Visual Studio.
2. Right-click the Models folder.
3. Select Add → Class.
4. Name the file Student.cs.
5. Add the following code:

```
namespace StudentRecordPWA.Models
{
   public class Student
   {
      public int Id { get; set; }

      public string Name { get; set; }

      public string Course { get; set; }

      public int Age { get; set; }
   }
}
```

Step 2: Create StudentsController.cs

1. Right-click the Controllers folder.
2. Select Add → Controller or Add → Class.
3. Name the file StudentsController.cs.

4. Add the following code:

```
using Microsoft.AspNetCore.Mvc;
using StudentRecordPWA.Models;
using System.Collections.Generic;

namespace StudentRecordPWA.Controllers
{
   public class StudentsController : Controller
   {
      public IActionResult Index()
      {
         List<Student> students = new List<Student>()
         {
            new Student { Id = 1, Name = "Uzma", Course = "BSc IT", Age = 20 },
            new Student { Id = 2, Name = "Nargis", Course = "BCA", Age = 21 },
            new Student { Id = 3, Name = "Sneha", Course = "BSc CS", Age = 19 }
         };

         return View(students);
      }
   }
}
```

Step 3: Create Index.cshtml

1. Open Views → Students.
2. If the Students folder does not exist, create it.
3. Create a new file named Index.cshtml.

4. Add the following code:

```
@model IEnumerable<StudentRecordPWA.Models.Student>

<h2>Student Records</h2>

<table border="1" cellpadding="10">
  <tr>
      <th>Id</th>
      <th>Name</th>
      <th>Course</th>
      <th>Age</th>
  </tr>

  @foreach (var item in Model)
  {
      <tr>
        <td>@item.Id</td>
        <td>@item.Name</td>
        <td>@item.Course</td>
        <td>@item.Age</td>
      </tr>
  }
</table>
```

Step 4: Create manifest.json

1. Open the wwwroot folder.
2. Create a new file named manifest.json.

3. Add the following code:

```
{
  "name": "Student Record PWA",
  "short_name": "StudentPWA",
  "start_url": "/Students",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Step 5: Create service-worker.js

1. Inside the wwwroot folder, create a JavaScript file named service-worker.js.
2. Add the following code:

```
const CACHE_NAME = "student-pwa-cache";

const urlsToCache = [
    "/Students"
];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
          .then(cache => cache.addAll(urlsToCache))
    );
});

self.addEventListener("fetch", event => {
    event.respondWith(
        fetch(event.request)
          .catch(() => caches.match(event.request))
    );
});
```

Step 6: Update Layout.cshtml

1. Open Views → Shared → _Layout.cshtml.
2. Add the manifest link inside the <head> section.
3. Add the service worker registration script before the closing </body> tag.

```
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <link rel="manifest" href="/manifest.json">
</head>

<script>
  if ('serviceWorker' in navigator) {
      navigator.serviceWorker.register('/service-worker.js')
        .then(() => console.log('Service Worker Registered'))
        .catch(error => console.log('Service Worker Error:', error));
  }
</script>
```

Keep the existing layout code and place these additions in the correct sections rather than replacing the entire layout file.

Step 7: Create the Icons Folder

1. Inside wwwroot, create a folder named icons.
2. Add icon1.png with dimensions 192 × 192.
3. Add icon2.png with dimensions 512 × 512.

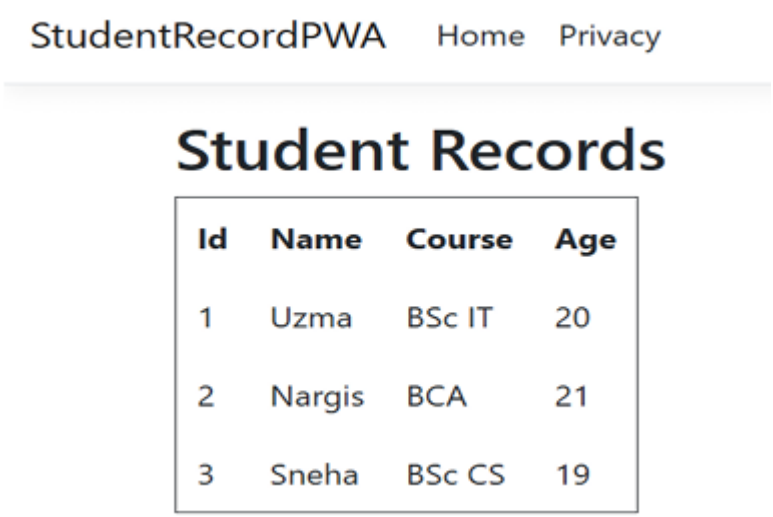
Step 8: Check the File Structure

StudentRecordPWA

Step 9: Run the Project

1. Run the project.
2. Open /Students in the browser.
3. View the student details in the table.

OUTPUT


StudentRecordPWA
Home
Privacy
Student Records
Id
Name
Course
Age
1
Uzma
BSc IT
20
2
Nargis
BCA
21
3
Sneha
BSc CS
19

Practical – 3 (a): Create a Basic Progressive Web Application

Step 1: Open Program.cs

1. Open your ASP.NET Core project in Visual Studio.
2. Open the Program.cs file.
3. Add the following code:

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

Step 2: Create Index.html

1. Open the wwwroot folder.
2. Create a new HTML file named index.html.
3. Add the following code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Basic PWA</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="manifest" href="manifest.json">
</head>
<body>
  <h1>My Progressive Web Application</h1>
  <p>This is the starting page of PWA.</p>
</body>
</html>
```

Step 3: Create manifest.json

1. Inside the wwwroot folder, create a new file named manifest.json.

2. Add the following code:

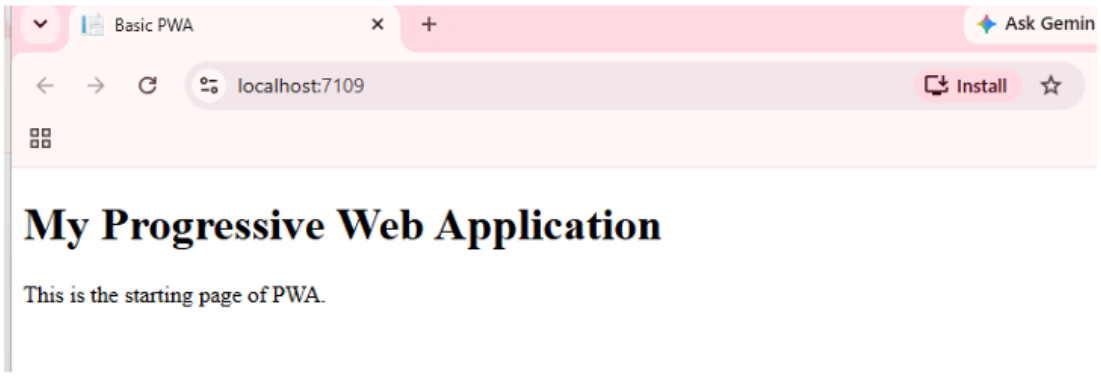
```
{
  "name": "Basic Progressive Web Application",
  "short_name": "BasicPWA",
  "description": "Simple PWA Example",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#000000",
  "orientation": "portrait",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Step 4: Create the Icons Folder

1. Inside wwwroot, create a folder named icons.
2. Add an image named icon1.png with dimensions 192 × 192.
3. Add an image named icon2.png with dimensions 512 × 512.

Step 6: Run the Project

OUTPUT



Practical – 3 (b): Create an ASP.NET Core Web API

Step 1: Create a Web API Project

1. Open Visual Studio.
2. Click Create a new project.
3. Select ASP.NET Core Web API.
4. Set the project name to StudentAPI.
5. Select the required .NET framework and click Create.

Step 2: Create the Models Folder

1. Right-click the project and select Add → New Folder.
2. Name the folder Models.
3. Create a class named Student.cs inside it.

Step 3: Write the Student Model Code

```
namespace StudentAPI.Models
{
  public class Student
  {
    public int Id { get; set; }
    public string Name { get; set; }
    public string Course { get; set; }
    public int Age { get; set; }
  }
}
```

Step 4: Create the API Controller

1. Right-click the Controllers folder.
2. Select Add → Controller.
3. Select API Controller - Empty.
4. Name it StudentController.cs.

Step 5: Write the API Controller Code

```
using Microsoft.AspNetCore.Mvc;
using StudentAPI.Models;

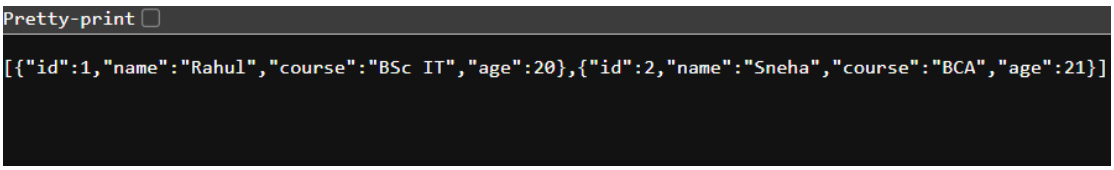
namespace StudentAPI.Controllers
{
   [ApiController]
   [Route("api/[controller]")]
   public class StudentController : ControllerBase
   {
      [HttpGet]
      public List<Student> GetStudents()
      {
         return new List<Student>
         {
            new Student
            {
               Id = 1,
               Name = "Rahul",
               Course = "BSc IT",
               Age = 20
            },
            new Student
            {
               Id = 2,
               Name = "Sneha",
               Course = "BCA",
               Age = 21
            }
         };
      }
   }
}
```

Step 6: Run the Application

Step 7: Test the API

https://localhost:7212/api/student

OUTPUT



Practical – 4 (a): Develop CURD Operation using the ASP.Net Core Web API And Entity Framework core

Step 1: Create ASP.NET Core Web API Project

1. Open Visual Studio
2. Click Create New Project
3. Select ASP.NET Core Web API
4. Project Name: StudentAPIWithEF
5. Click Next
6. Select: .NET 8 and Enable OpenAPI Support (in Extra info Page)
7. Click Create

Step 2: Install Entity Framework Core Packages

Go to: Tools → NuGet Package Manager → Manage NuGet Packages

Install these packages:

1. Microsoft.EntityFrameworkCore
2. Microsoft.EntityFrameworkCore.SqlServer
3. Microsoft.EntityFrameworkCore.Tools
4. Swashbuckle.AspNetCore
5. Microsoft.EntityFrameworkCore.Design

Step 3: Create Model Class

Right Click on Project Name and Create folder: Models Create file under Models Folder: Student.cs
Write code in the file

Step 4: Create DbContext Class

Right Click on Project Name and Create folder: Data Under Data Folder Create file: AppDbContext.cs

Write following code in file

Step 5: Add LocalDB Connection String

In Solution Explorer Open: appsettings.json file

Add Following code:

Step 6: Configure Database in Program.cs

In Solution explorer Open: Program.cs Replace with:

Step 7: Create Controller

If not existing the Right Click on Project Name and Create folder: Controllers Inside Controller folder Create and add class file: StudentsController.cs

Add following code:

```
using Microsoft.AspNetCore.Mvc;
using StudentAPIWithEF.Data;
using StudentAPIWithEF.Models;
```

```
namespace StudentAPIWithEF.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly AppDbContext _context;

        public StudentsController(AppDbContext context)
        {
            _context = context;
        }

        // GET ALL STUDENTS
        [HttpGet]
        public IActionResult GetStudents()
        {
            return Ok(_context.Students.ToList());
        }

        // GET STUDENT BY ID
        [HttpGet("{id}")]
        public IActionResult GetStudent(int id)
        {
            var student = _context.Students.Find(id);

            if (student == null)
                return NotFound();

            return Ok(student);
        }

        // INSERT STUDENT
        [HttpPost]
        public IActionResult AddStudent(Student student)
        {
            _context.Students.Add(student);
            _context.SaveChanges();

            return Ok(student);
        }

        // UPDATE STUDENT
        [HttpPut("{id}")]
        public IActionResult UpdateStudent(int id, Student student)
        {
            if (id != student.Id)
                return BadRequest("ID does not match.");
```

```
        var data = _context.Students.Find(id);

        if (data == null)
            return NotFound();

        data.Name = student.Name;
        data.Age = student.Age;
        data.Course = student.Course;

        _context.SaveChanges();

        return Ok(data);
    }

    // DELETE STUDENT
    [HttpDelete("{id}")]
    public IActionResult DeleteStudent(int id)
    {
        var student = _context.Students.Find(id);

        if (student == null)
            return NotFound();

        _context.Students.Remove(student);
        _context.SaveChanges();

        return Ok("Student deleted successfully.");
    }
  }
}
```

Step 8: Create Database using Migration

Open

Tools → NuGet Package Manager → Package Manager Console

Run:

Add-Migration InitialCreate

then Run:-

Update-Database

This will automatically create:

- StudentDB database
- Students table

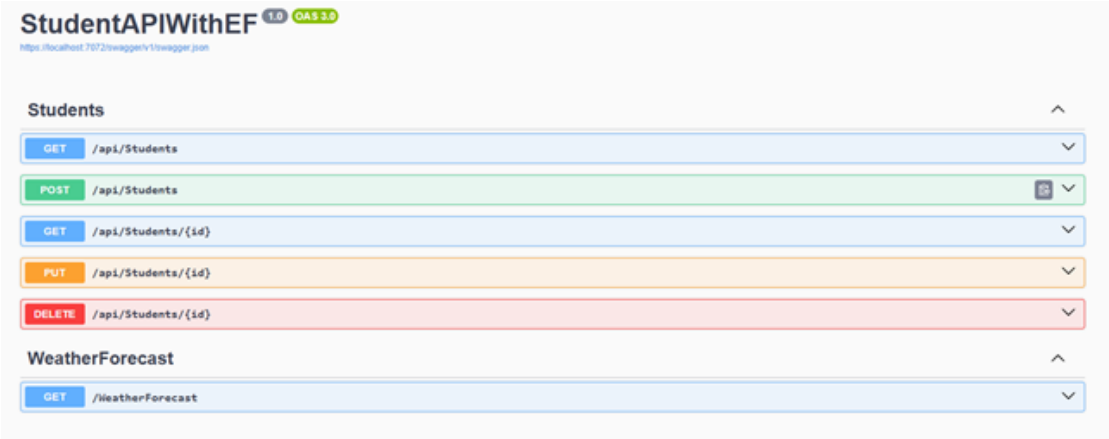
inside SQL Server LocalDB.

Step 9: Run Project

Swagger UI will open automatically.

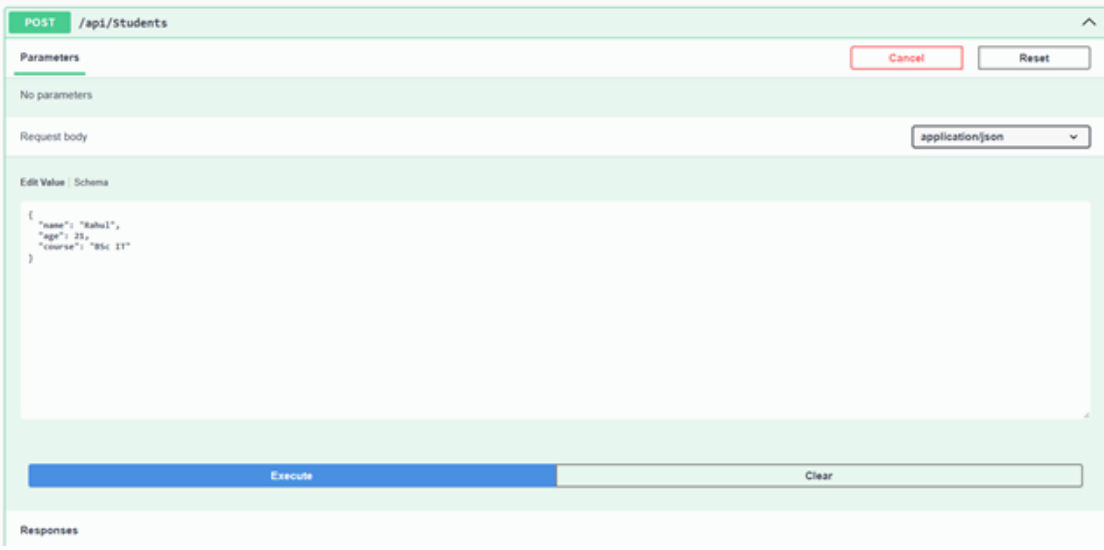
Example:

https://localhost:7288/swagger



Step 10: Test CRUD Operations

```
{
  "name": "Rahul",
  "age": 21,
  "course": "BSc IT"
}
```



Get→ View All Students

/api/students

GET BY ID

/api/students/1

PUT → Update Student

```
{
  "name": "Rahul",
  "age": 21,
  "course": "MCA "
}
```

DELETE Student /api/students/1

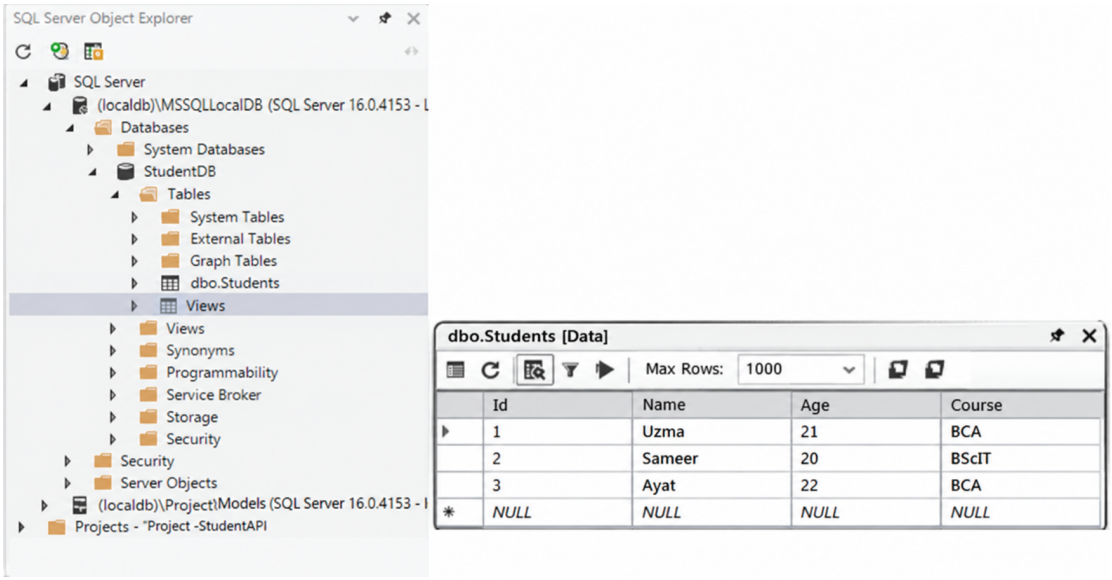
Step 11: View Database in Visual Studio

Go to: View → SQL Server Object Explorer

Open: (localdb)\MSSQLLocalDB

Then: Databases → StudentDB → Tables

You can view inserted student records directly.



Practical – 4 (b): Create an API endpoint to calculate Factorial of a Number and test it using Postman.

Step 1: Create ASP.NET Core Web API Project

1. Open Visual Studio
2. Create New Project
3. Select: ASP.NET Core Web API
4. Project Name: FactorialAPI
5. Select: .NET 8 Enable OpenAPI Support
6. Click Create

Step 2: Create Controller

In Solution explorer Create a class file: Controllers/MathController.cs

Add code:

```
using Microsoft.AspNetCore.Mvc;

namespace FactorialAPI.Controllers
{
   [Route("api/[controller]")]
   [ApiController]
   public class MathController : ControllerBase
   {
      [HttpGet("factorial/{number}")]
      public IActionResult GetFactorial(int number)
      {
         if (number < 0)
         {
            return BadRequest(
               "Factorial not possible for negative numbers.");
         }

         long factorial = 1;

         for (int i = 1; i <= number; i++)
         {
            factorial *= i;
         }

         return Ok(new
         {
            Number = number,
            Factorial = factorial
         });
      }
   }
}
```

Step 3: Run the Project

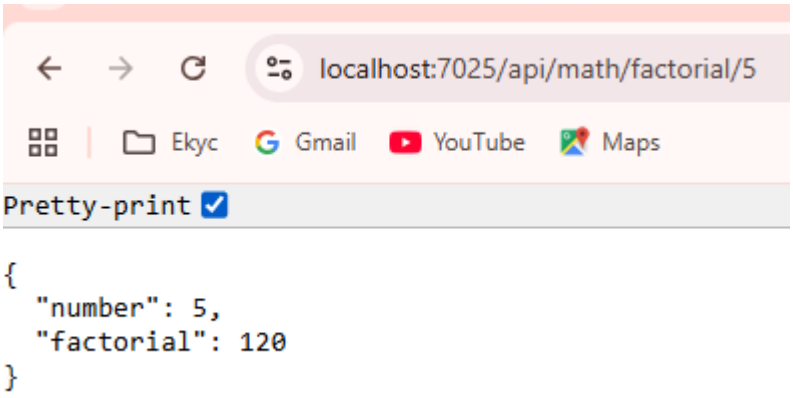
Project will run. Example URL: https://localhost:7282

Step 4: Test API in Browser

Open:-

https://localhost:7282/api/math/factorial/5

Output:-



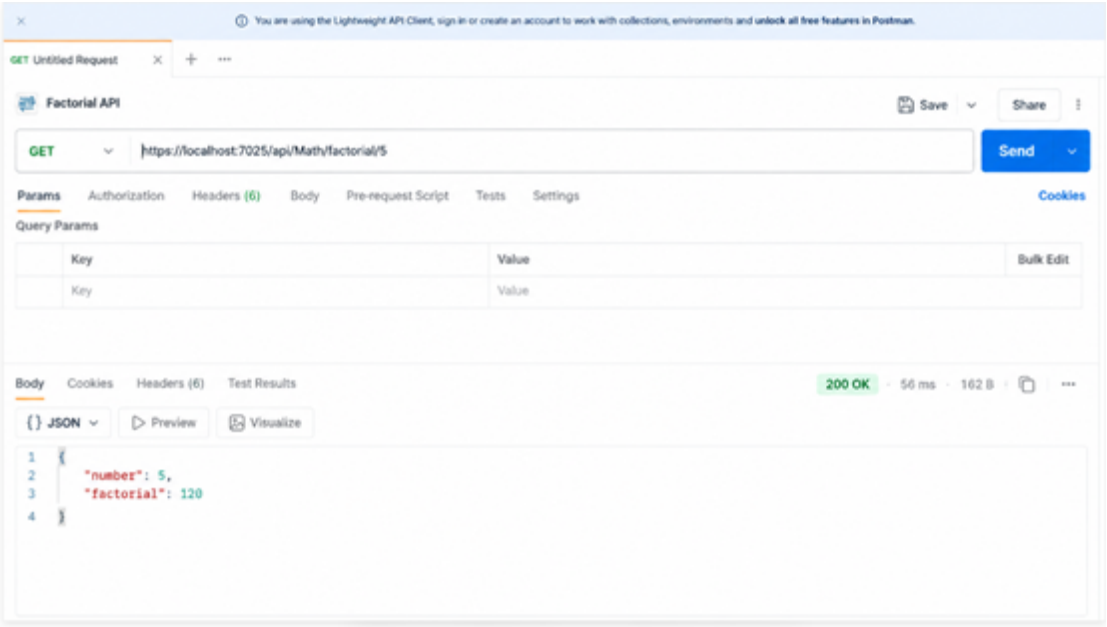
Step 5: Test using Postman

Install Postman

Open Postman (https://dl.pstmn.io/download/latest/win64)

Select: GET Enter URL: https://localhost:7282/api/math/factorial/5

Click: Send

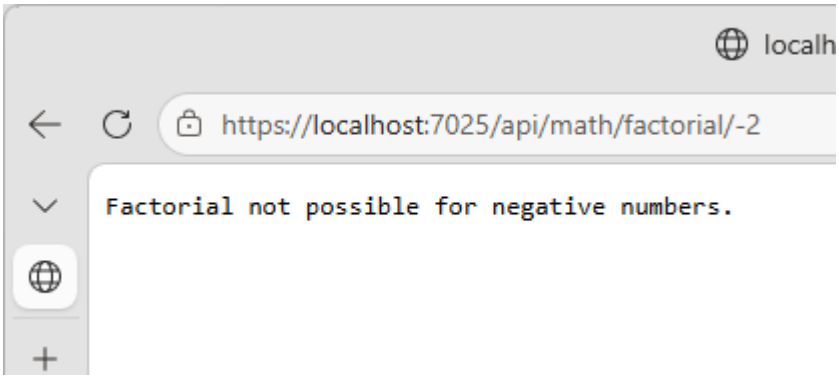


Negative number Test:-

URL:-

localhost:7025/api/math/factorial/-2

Output:-



Practical – 5 (a): Create a PWA with a Web App Manifest File

- App
- name
- Icons
- Start URL
- Display mode
- Implement Push Notifications in a PWA.

Step 1: Create Project

Create a simple ASP.NET Core Empty Web App

Project Name: PushNotification

Step 2: Create Manifest File

In Solution explorer Create new folder: wwwroot

Inside wwwroot folder Create JSON file: manifest.json

Add code:

```
{
  "name": "My PWA Application",
  "short_name": "MyPWA",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Step 3:- Add Icons

Create a floder under wwwroot as :

Wwwroot/icons

Download and add small icon images in floder or paste

- Icon1.png
- Icon2.png

Step 4:- Create HTML page

Create file inside wwwroot floder:

Wwwroot/index.html

Add code:

```
<!DOCTYPE html>
<html>
<head>
  <title>My PWA</title>
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#0d6efd">
</head>
<body>
  <h1>My Progressive Web App</h1>

  <button onclick="showNotification()">
    Show Notification
  </button>

  <script src="app.js"></script>
</body>
</html>
```

Step 5:- Create Javascript File

In Solution explorer Create JavaScript file:

wwwroot/app.js Add code:

```
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('service-worker.js')
        .then(() => console.log('Service Worker Registered'));
}

function showNotification() {
    Notification.requestPermission().then(permission => {
        if (permission === 'granted') {
            navigator.serviceWorker.ready.then(registration => {
                registration.showNotification('PWA Notification', {
                    body: 'Push Notification Working Successfully!',
                    icon: 'icons/icon1.png'
                });
            });
        }
    });
}
```

Step 6: Create Service Worker

Create JavaScript File:

wwwroot/service-worker.js

Add code:

```
self.addEventListener('install', event => {
   console.log('Service Worker Installed');
});

self.addEventListener('activate', event => {
   console.log('Service Worker Activated');
});
```

Step 7: Configure Program.cs

In Solution Explorer Open: Program.cs

Add code:

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

Step 8: Run Application

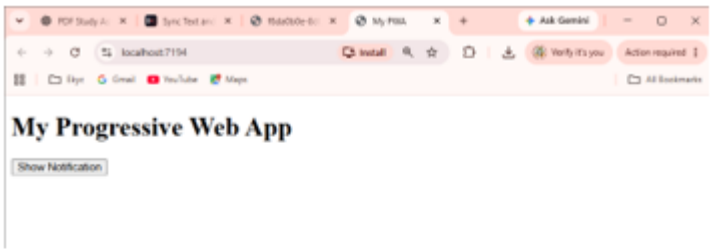
Browser opens URL Somewhat Similar like:

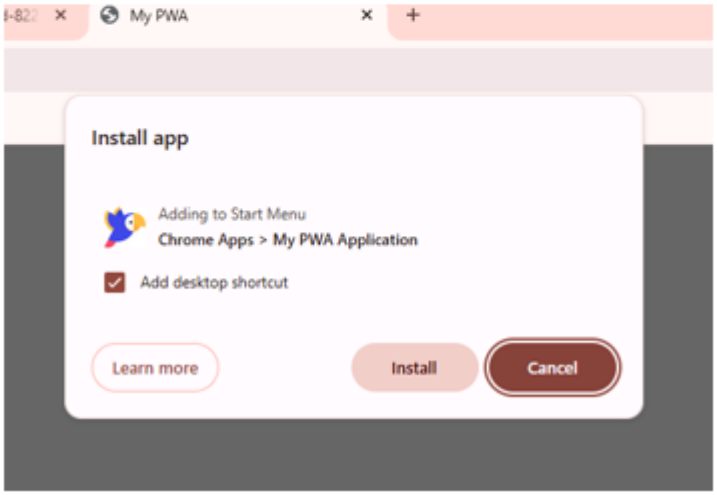
https://localhost:7194

Step 9: Install PWA

In Chrome:

1. Open website
2. Click Install icon in address bar
3. App installs like mobile app

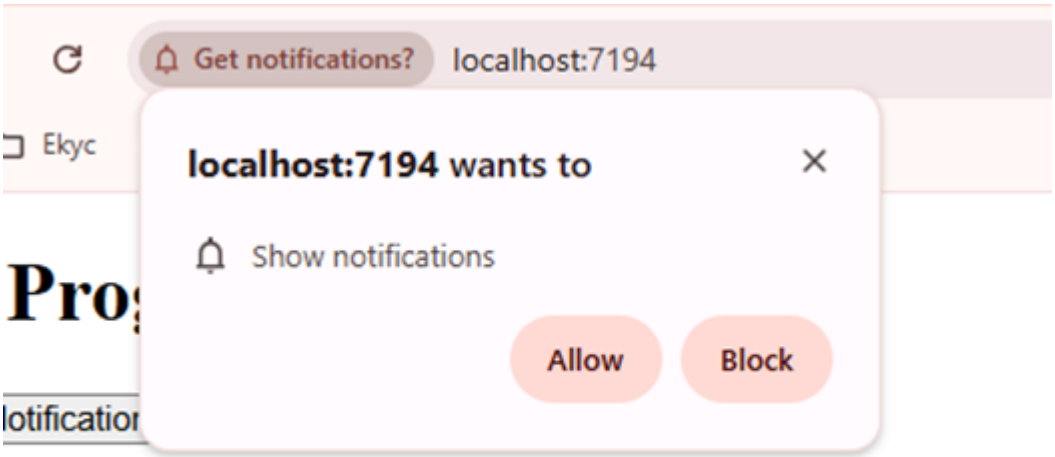




Step 10: Test Push Notification

Click: Show Notification

Browser notification will appear.



Practical – 5 (b): Develop an E-Commerce PWA with product listing and offline cart functionality.

Step 1: Create Project

Create a simple ASP.NET Core Empty Web App.

Project Name: ECommercePWA

Step 2: Configure Program.cs

In Solution explorer Open:

Program.cs

Replace the code with following code:

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

Step 3: Create Manifest File

In Solution explorer Right Click Project Name and Create wwwroot folder.

Inside the folder create JSON file. wwwroot/manifest.json

Add following code in the file:

```
{
  "name": "E-Commerce PWA",
  "short_name": "ShopPWA",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Step 4: Add icons

In Solution explorer Create folder: wwwroot/icons

Download and Add Small icon file in same folder:

- icon1.png
- icon2.png

Step 5 : Create HTML Page

Under wwwroot folder Create HTML file:

wwwroot/index.html Add:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>E-Commerce PWA</title>

  <link rel="manifest" href="/manifest.json">

  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      background-color: #f4f4f4;
    }

    h1, h2 {
      color: #333;
    }

    .product {
      background: white;
      padding: 15px;
      margin: 10px;
      display: inline-block;
      width: 200px;
      border-radius: 8px;
    }

    button {
      background-color: #0d6efd;
      color: white;
      border: none;
      padding: 10px;
      cursor: pointer;
      border-radius: 5px;
```

```
    }

    #cart {
      background: white;
      padding: 15px;
      margin-bottom: 20px;
      border-radius: 8px;
    }
  </style>
</head>

<body>

  <h1>E-Commerce PWA</h1>

  <!-- Cart Section -->
  <div id="cart">
    <h2>Cart Items</h2>
    <ul id="cartItems">
      <li>No items in cart</li>
    </ul>
    <h3 id="total">Total: ₹0</h3>
  </div>

  <h2>Products</h2>

  <div class="product">
    <h3>Laptop</h3>
    <p>Price: ₹50000</p>
    <button onclick="addToCart('Laptop', 50000)">
      Add to Cart
    </button>
  </div>

  <div class="product">
    <h3>Mobile</h3>
    <p>Price: ₹20000</p>
    <button onclick="addToCart('Mobile', 20000)">
      Add to Cart
    </button>
  </div>

  <div class="product">
    <h3>Headphones</h3>
    <p>Price: ₹3000</p>
    <button onclick="addToCart('Headphones', 3000)">
      Add to Cart
    </button>
```

```
  </div>

  <script src="/script.js"></script>

</body>
</html>
```

Step 6: Create JavaScript File

Under wwwroot folder Create JavaScript File:

wwwroot/app.js

Add Following code in file:

```
let cart = [];

function addToCart(name, price) {
    cart.push({
        name: name,
        price: price
    });

    displayCart();
}

function displayCart() {
    let cartItems = document.getElementById("cartItems");
    let total = document.getElementById("total");

    cartItems.innerHTML = "";

    let sum = 0;

    if (cart.length === 0) {
        cartItems.innerHTML = "<li>No items in cart</li>";
    }

    cart.forEach(function(item) {
        let li = document.createElement("li");

        li.textContent = item.name + " - ₹" + item.price;

        cartItems.appendChild(li);

        sum += item.price;
    });

    total.textContent = "Total: ₹" + sum;
}
```

Step 7: Create Service Worker

Create JavaScript File: wwwroot/service-worker.js

Add Following code:

```
const CACHE_NAME = 'ecommerce-cache-v1';

const urlsToCache = [
  '/',
  '/index.html',
  '/script.js',
  '/manifest.json'
];

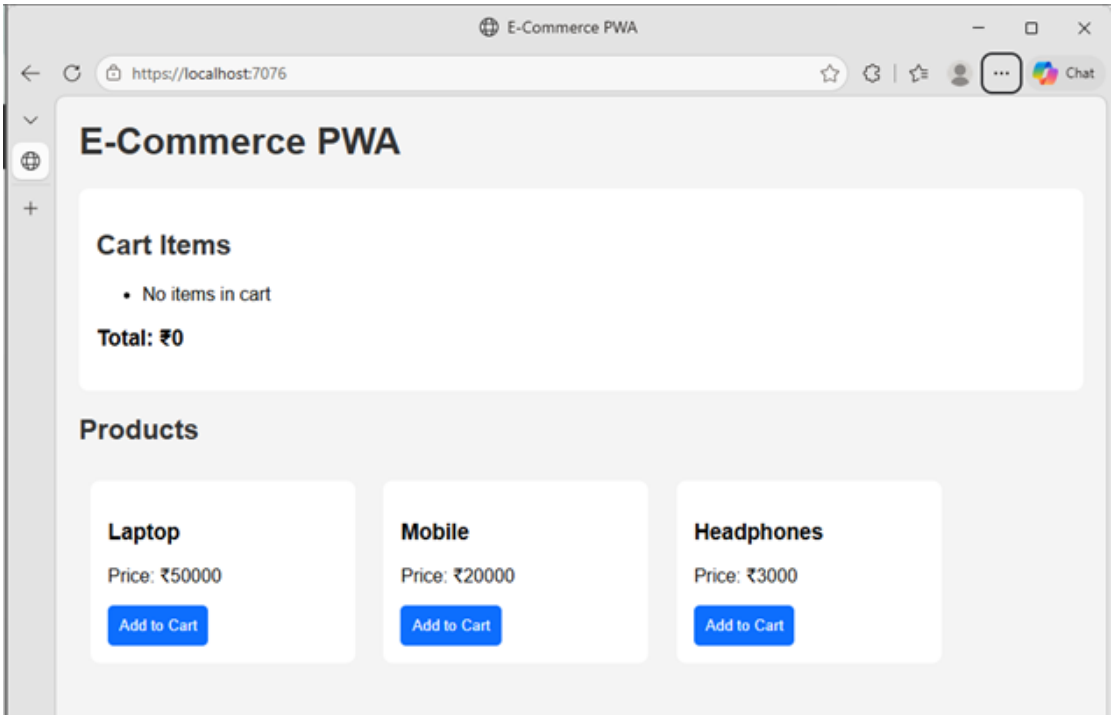
// Install Service Worker
self.addEventListener('install', event => {
   event.waitUntil(
     caches.open(CACHE_NAME)
       .then(cache => {
           return cache.addAll(urlsToCache);
         })
     );
});

// Fetch cached files
self.addEventListener('fetch', event => {
   event.respondWith(
      caches.match(event.request)
       .then(response => {
           return response || fetch(event.request);
       })
   );
});
```

Step 8: Run Application

Browser opens URL Similar like:

https://localhost:7076



Step 9 : Test Offline Functionality

1. Open Chrome DevTools
2. Go to: Network → Offline
3. Refresh page

Application still works because:

- Files cached by Service Worker
- Cart stored locally

